

Лабораторная работа №5

**Моделирование параллельных систем с помощью сетей Петри
на примере задачи об обедающих философах**

Еремина Оксана Андреевна

Содержание

1	Цель работы	6
2	Теоретическое введение	7
2.1	Сети Петри	7
2.2	Задача об обедающих философх	8
2.3	Алгоритм Гиллеспи	8
3	Выполнение лабораторной работы	9
4	Моделирование задачи об обедающих философх с помощью сетей Петри	11
4.1	Введение	11
4.2	Загрузка необходимых пакетов	11
4.3	Параметры моделирования	12
4.4	Выводы	15
5	Анимация работы сети Петри для задачи об обедающих философх	17
5.1	Введение	17
5.2	Загрузка пакетов	17
5.3	Параметры	17
5.4	Построение классической сети	18
5.5	Стохастическая симуляция	18
5.6	Создание анимации	18
5.7	Сохранение анимации	19
5.8	Интерпретация	19
6	Сравнительный анализ классической сети Петри и сети с арбитром	20
6.1	Введение	20
6.2	Загрузка пакетов	20
6.3	Загрузка данных	20
6.4	Параметры	21
6.5	Построение графиков для классической сети	21
6.6	Построение графиков для сети с арбитром	21
6.7	Объединение графиков	22
6.8	Интерпретация результатов	22
6.9	Выводы	24

7 Вывод	25
Список литературы	26

Список иллюстраций

3.1	Запуск кода	9
3.2	Запуск кода	9
3.3	Запуск кода	9
3.4	Запуск кода	9
3.5	Компиляция dining_philosophers.jl	10
3.6	Компиляция dining_philosophers_animation.jl	10
3.7	Компиляция dining_philosophers_report.jl	10
4.1	График	13
4.2	График	15
6.1	График	23

Список таблиц

1 Цель работы

1. Построить сеть Петри для задачи об обедающих философах.
2. Обнаружить deadlock в классической модели.
3. Модифицировать сеть введением арбитра для предотвращения deadlock.
4. Провести стохастическое моделирование с помощью алгоритма Гиллеспи.
5. Сравнить поведение двух моделей и визуализировать результаты.

2 Теоретическое введение

2.1 Сети Петри

Сеть Петри — математический аппарат для моделирования дискретных систем с параллелизмом и синхронизацией. Формально определяется как четвёрка:

$$N = (P, T, F, M_0),$$

где:

- P — множество позиций (состояния, ресурсы);
- T — множество переходов (события, действия);
- F — множество дуг между позициями и переходами;
- M_0 — начальная маркировка (распределение фишек).

Графически позиции обозначаются кругами, переходы — прямоугольниками. Фишки внутри позиций показывают текущее состояние системы. Переход срабатывает, если во всех входных позициях достаточно фишек. При срабатывании фишки изымаются из входных позиций и добавляются в выходные.

Ключевые свойства сетей Петри:

- **Ограниченность** — фишек в позиции не больше заданного предела.
- **Активность** — отсутствие тупиков, каждый переход может сработать.
- **Достижимость** — возможность попасть из начального состояния в целевое.

2.2 Задача об обедающих философах

Сформулирована Эдсгером Дейкстрой в 1965 году для иллюстрации проблем синхронизации в параллельных системах.

Постановка:

- N философов сидят за круглым столом.
- Между соседями лежит одна вилка (всего N вилок).
- Философ может думать, быть голодным или есть.
- Чтобы поест, нужны две вилки — слева и справа.

Проблема deadlock:

Если каждый философ возьмёт левую вилку и будет ждать правую, все заблокируются — система замирает.

Решение с арбитром:

Вводится дополнительный ресурс (арбитр), ограничивающий число одновременно едящих философов до $N-1$, что предотвращает deadlock.

2.3 Алгоритм Гиллеспи

Стохастический метод моделирования, где время до следующего события и выбор перехода определяются случайным образом на основе интенсивностей переходов.

3 Выполнение лабораторной работы

Создала необходимые файлы, куда скопировала весь код, предоставленный в лабораторной работе.

Запустила их всех, сначала пробежав `install_packages.jl`, установив необходимые библиотеки (рис. 3.1, рис. 3.2, рис. 3.3, рис. 3.4).

```
deling/labs/lab05/project$ julia --project=. scripts/install_packages.jl
Updating registry at `~/.julia/registries/General.toml`
Resolving package versions...
Updating `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/p
project/Project.toml`
^ [1dea7af3] + OrdinaryDiffEq v6.105.0
Manifest No packages added to or removed from `~/work/study/2026--1/2026-1-s
tudy-simulation-modeling/labs/lab05/project/Manifest.toml`
```

Рисунок 3.1: Запуск кода

```
deling/labs/lab05/project$ julia --project=. scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
```

Рисунок 3.2: Запуск кода

```
deling/labs/lab05/project$ julia --project=. scripts/dining_philosophers_animati
on.jl
Could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/zrdagdelen/work/study/2026--1/2026-1-study-simu
lation-modeling/labs/lab05/project/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 3.3: Запуск кода

```
deling/labs/lab05/project$ julia --project=. scripts/dining_philosophers_report.
jl
Отчёт сохранён в plots/final_report.png
Could not create decoration from factory! Running with no decorations.
```

Рисунок 3.4: Запуск кода

Далее создала литературный код для всех основных файлов. После скомпилировала чистый код, jupyter notebook и quarto с помощью файла с кодом `tangle.jl` (рис. 3.5), (рис. 3.6), (рис. 3.7).

```
deling/labs/lab05/project$ julia --project=. scripts/tangle.jl scripts/dining_philosophers.jl
Генерация из: scripts/dining_philosophers.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers/dining_philosophers.jl`
```

Рисунок 3.5: Компиляция `dining_philosophers.jl`

```
deling/labs/lab05/project$ julia --project=. scripts/tangle.jl scripts/dining_philosophers_animation.jl
Генерация из: scripts/dining_philosophers_animation.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_animation/dining_philosophers_animation.jl`
```

Рисунок 3.6: Компиляция `dining_philosophers_animation.jl`

```
zrdagdelen@zrdagdelen-MacBookPro:~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/tangle.jl scripts/dining_philosophers_report.jl
Генерация из: scripts/dining_philosophers_report.jl
[ Info: generating plain script file from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl`
✓ Чистый скрипт: /home/zrdagdelen/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report/dining_philosophers_report.jl
[ Info: generating markdown page from `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/work/study/2026--1/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining_philosophers_report/dining_philosophers_report.qmd`
```

Рисунок 3.7: Компиляция `dining_philosophers_report.jl`

Кад из файлов и анализ результатов предоставлен ниже.

4 Моделирование задачи об обедающих философах с помощью сетей Петри

4.1 Введение

Данный скрипт выполняет стохастическое моделирование двух вариантов сети Петри для классической задачи об обедающих философах: 1. Классическая модель (без арбитра) — подвержена deadlock 2. Модифицированная модель с арбитром — предотвращает deadlock

4.2 Загрузка необходимых пакетов

```
using DrWatson
@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using DataFrames, CSV, Plots
```

4.3 Параметры моделирования

- $N = 5$ — количество философов
- $t_{\max} = 50.0$ — время моделирования

```
N = 5  
tmax = 50.0
```

4.3.1 Моделирование классической сети (без арбитра)

В классической постановке каждый философ пытается взять сначала левую вилку, затем правую. Это может привести к взаимной блокировке (deadlock).

```
println("=== 🍴🍴🍴🍴🍴 🍴 (🍴 🍴🍴🍴🍴) ===")  
net_classic, u0_classic, _ = build_classical_network(N)
```

4.3.2 Стохастическая симуляция

Используется алгоритм Гиллеспи для стохастического моделирования.

```
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

4.3.3 Сохранение результатов

Результаты сохраняются в формате CSV для дальнейшего анализа.

```
CSV.write(datadir("dining_classic.csv"), df_classic)
```

4.3.4 Проверка наличия deadlock

```
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock ██████████: $dead")
```

4.3.5 Визуализация эволюции маркировки

Строятся графики для всех позиций сети: Think, Hungry, Eat, Fork (рис. 4.1).

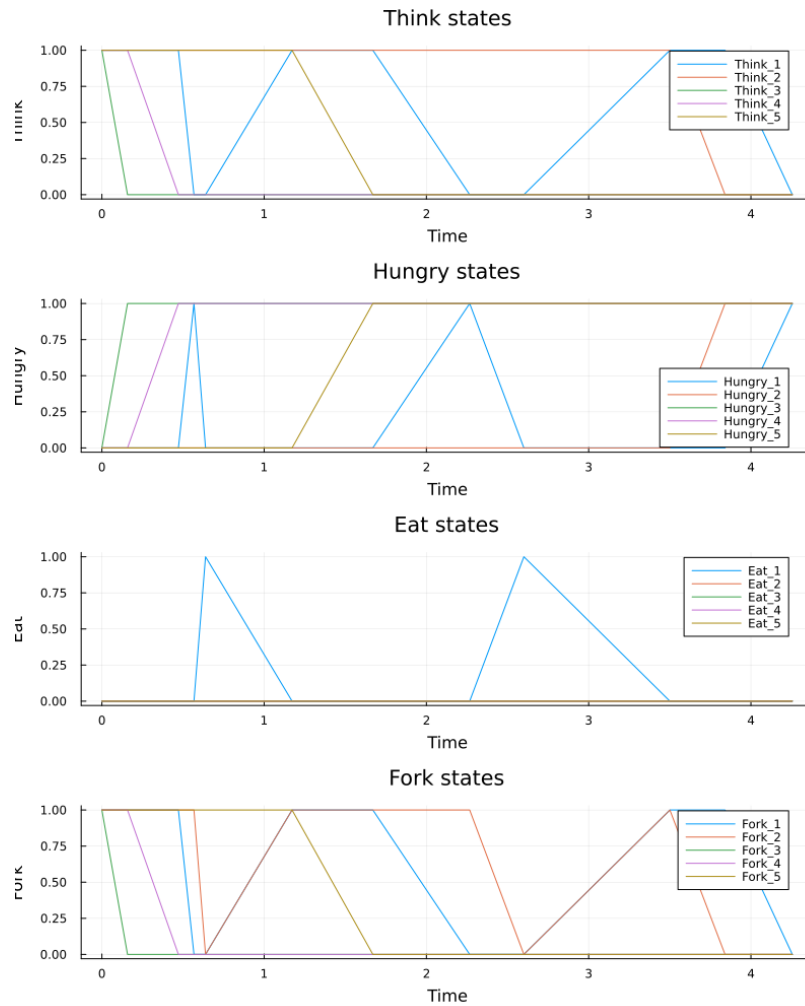


Рисунок 4.1: График

```
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir("classic_simulation.png"))
```

```
## 10000 iterations, 1000 samples, 10000 samples

# 10000 iterations, 1000 samples, 10000 samples - 10000 samples,
# 10000 iterations, 1000 samples, 10000 samples.

```julia
println("\n=== 10000 samples ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
```

### 4.3.6 Стохастическая симуляция

```
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
```

### 4.3.7 Сохранение результатов

```
CSV.write(datadir("dining_arbiter.csv"), df_arb)
```

### 4.3.8 Проверка наличия deadlock

```
dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock 10000 samples: $dead_arb")
```

### 4.3.9 Визуализация эволюции маркировки

```
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir("arbiter_simulation.png"))
```

Посмотрим на график (рис. 4.2)

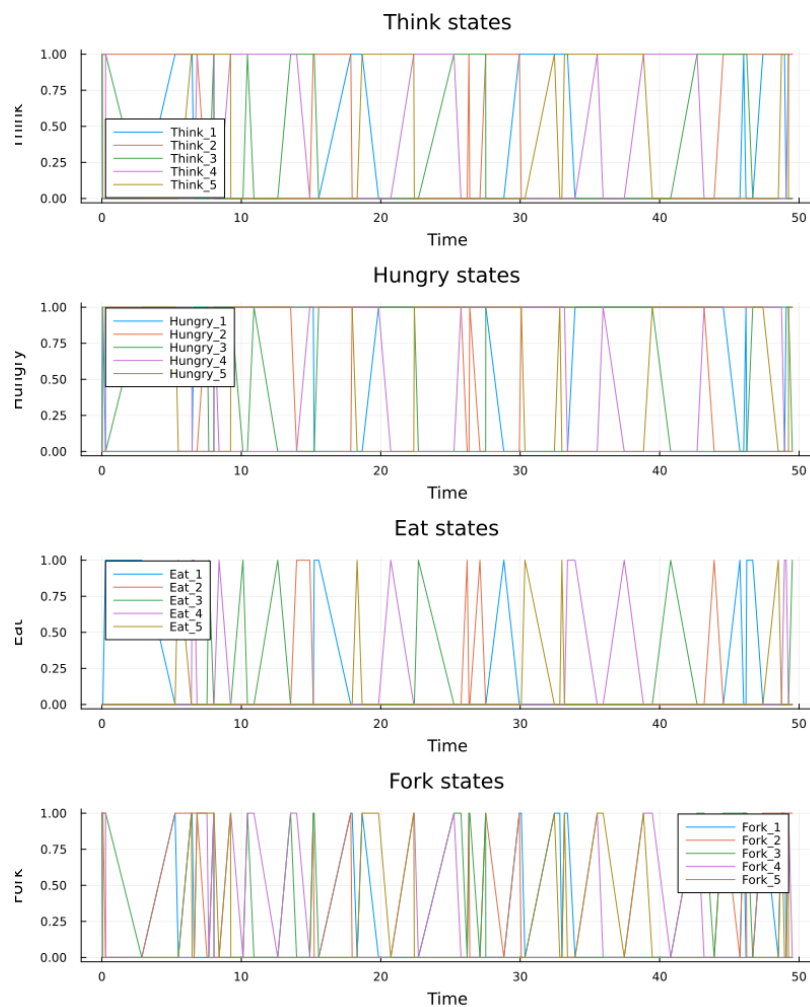


Рисунок 4.2: График

## 4.4 Выводы

- Классическая сеть демонстрирует deadlock (философы застревают в состоянии Hungry)

- Сеть с арбитром успешно предотвращает deadlock (система продолжает работать)

Результаты сохранены в: - `data/dining_classic.csv` и `data/dining_arbiter.csv`  
— траектории состояний - `plots/classic_simulation.png` и `plots/arbiter_simulation.png`  
— графики



## 5 Анимация работы сети Петри для задачи об обедающих философах

### 5.1 Введение

Данный скрипт создаёт анимацию, показывающую динамику изменения маркировки в классической сети Петри. Анимация позволяет наглядно увидеть, как фишки перемещаются между позициями и как возникает deadlock.

### 5.2 Загрузка пакетов

```
using DrWatson
@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers
using Plots, Random
```

### 5.3 Параметры

- $N = 3$  — количество философов (меньше для лучшей визуализации)
- $t_{\max} = 30.0$  — время моделирования

```
N = 3
tmax = 30.0
```

## 5.4 Построение классической сети

```
net, u0, names = build_classical_network(N)
```

## 5.5 Стохастическая симуляция

Фиксируем seed для воспроизводимости результатов.

```
Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
```

## 5.6 Создание анимации

Каждый кадр анимации представляет собой столбчатую диаграмму текущей маркировки. По оси X — позиции сети, по оси Y — количество фишек.

```
anim = @animate for row in eachrow(df)
 u = [row[col] for col in propertynames(row) if col != :time]
 bar(
 1:length(u), u,
 legend = false,
 ylims = (0, maximum(u0) + 1),
 xlabel = "XXXXXX",
 ylabel = "XXXXX",
 title = "XXXXX = $(round(row.time, digits=2))",
)
end
```

```
)
 xticks!(1:length(u), string.(names), rotation = 45)
end
```

## 5.7 Сохранение анимации

Анимация сохраняется в формате GIF.

```
gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("XXXXXXXX XXXXXXXXXXXX ✕ plots/philosophers_simulation.gif")
```

## 5.8 Интерпретация

На анимации можно наблюдать: - Перемещение фишек между позициями Think, Hungry, Eat и Fork - В определённый момент в классической модели наступает deadlock — маркировка перестаёт изменяться, все философы застревают в состоянии Hungry

Это наглядно демонстрирует проблему взаимной блокировки в параллельных системах.

## 6 Сравнительный анализ

### классической сети Петри и сети с арбитром

#### 6.1 Введение

Данный скрипт строит сравнительный отчёт по двум моделям задачи об обедающих философах. Анализируется ключевой показатель — количество философов в состоянии «Ест» (Eat) в каждый момент времени.

#### 6.2 Загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DataFrames, CSV, Plots
```

#### 6.3 Загрузка данных

Используются ранее сохранённые результаты моделирования.

```
df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
```

## 6.4 Параметры

```
N = 5
```

## 6.5 Построение графиков для классической сети

Показываем динамику состояния Eat для каждого философа.

```
p1 = plot(
 xlabel = "Time",
 ylabel = "Eat (1/0)",
 title = "Dining Classic (5 Philosophers)",
 legend = :topright,
 legendtitle = "Eat",
)

for i = 1:N
 col = Symbol("Eat_$i")
 plot!(p1, df_classic.time, df_classic[:, col], label = "$i")
end
```

## 6.6 Построение графиков для сети с арбитром

```

p2 = plot(
 xlabel = "Time",
 ylabel = "Eaten (1/0)",
 title = "Eaten vs Time",
 legend = :topright,
 legendtitle = "Legend",
)

for i = 1:N
 col = Symbol("Eat_$i")
 plot!(p2, df_arbiter.time, df_arbiter[:, col], label = "$i")
end

```

## 6.7 Объединение графиков

Размещаем оба графика один под другим для удобного сравнения.

```

p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("final_report.png"))

println("Saved final_report.png to plots/final_report.png")

```

## 6.8 Интерпретация результатов

Посмотрим на график (рис. 6.1)

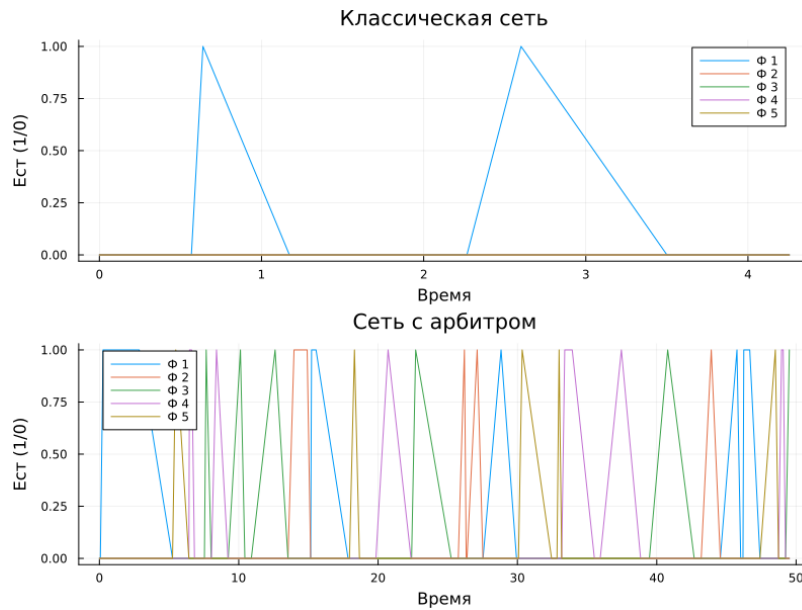


Рисунок 6.1: График

### 6.8.1 Классическая сеть (верхний график)

- Через некоторое время все линии  $Eat_i$  падают до нуля и остаются на нуле
- Это свидетельствует о наступлении deadlock — философы больше не могут есть
- Система полностью блокируется

### 6.8.2 Сеть с арбитром (нижний график)

- Линии  $Eat_i$  колеблются на протяжении всего времени моделирования
- Всегда есть хотя бы один философ, который ест
- Отсутствуют длительные периоды бездействия всех философов
- Система остаётся «живой» и не блокируется

## 6.9 Выводы

1. Классическая постановка задачи обедающих философов неизбежно приводит к deadlock
2. Введение арбитра (дополнительной позиции с N-1 фишками) эффективно предотвращает взаимную блокировку
3. Сети Петри являются удобным инструментом для моделирования и анализа проблем синхронизации в параллельных системах



## 7 Вывод

Сети Петри являются удобным и наглядным инструментом для моделирования параллельных систем. Задача об обедающих философах наглядно демонстрирует проблему deadlock, а модификация с арбитром показывает эффективный способ её решения. Стохастическое моделирование позволяет исследовать динамику системы в непрерывном времени.

# Список литературы

- Лабораторная №5